

"JSON é mais eficiente em termos de tempo de serialização/desserialização e tamanho de dados gerados do que XML, num cenário de microserviços."

João Paiva, Salomé Costa

Universidade de Coimbra, DEI

Emails: {uc2021216669,uc2021218601}@student.dei.uc.pt

Abstract - Este trabalho tem como objetivo investigar a eficiência dos formatos de dados JSON e XML em termos de tempo de serialização/desserialização e tamanho de dados gerados num cenário de microserviços. A hipótese é que o JSON, por ser um formato mais simples e compacto, será mais eficiente do que o XML, tanto em termos de rapidez de

processamento quanto em tamanho de ficheiros gerados durante a serialização. Para validar esta hipótese, iremos realizar um conjunto de testes experimentais em dois computadores distintos, medindo o tempo necessário para serializar e desserializar os dados, bem como o tamanho dos dados gerados em ambos os formatos (JSON e XML).

1. INTRODUÇÃO

Este trabalho tem como objetivo comparar a eficiência dos formatos de dados JSON e XML em termos de rapidez de serialização/desserialização e tamanho dos dados gerados, no contexto de microserviços. A hipótese central é que o JSON, por ser um formato mais leve e simples, será mais rápido que o XML, que é mais robusto, mas mais pesado. Conforme apresentado nos slides Teóricos de JSON, este formato destaca-se pela sua simplicidade e baixo overhead, facilitando a troca de dados entre serviços. Em contraste, o XML permite uma estruturação de dados mais rica, mas com maior complexidade, como mencionado também nos slides de XML.

Os testes propostos serão realizados em dois computadores, focando-se apenas nos tempos de serialização e desserialização e no tamanho dos ficheiros gerados, sem envolver comunicação em rede. Esta análise será fundamental para determinar qual dos formatos oferece maior rapidez em termos de serialização/desserialização e menor tamanho de ficheiros gerados em cenários de microserviços.

2. CONCEITOS

2.1. Microserviços

Os microserviços são um estilo de arquitetura de aplicações em que um conjunto independente de serviços são responsáveis por funcionalidades específicas e se comunicam por meio de APIs ou mensagens. Para que a comunicação entre microserviços seja eficiente, os dados devem ser serializados em formatos que sejam compatíveis com múltiplas plataformas e linguagens, como JSON e XML. Estes formatos permitem a troca de informações de forma padronizada, mas com

diferenças no desempenho e no tamanho dos dados gerados.

2.2. JSON (JavaScript Object Notation)

O JSON é utilizado para estruturar dados em formato de texto e permitir a troca de dados entre aplicações de forma simples, leve e rápida. Foi criado como uma solução para transmitir dados entre sistemas de forma mais eficiente do que o XML, especialmente em aplicações orientadas a dados. JSON usa uma estrutura baseada em pares nome/valor e arrays, o que facilita a sua serialização. Devido à sua simplicidade, o JSON produz ficheiros menores e é mais fácil de processar por máquinas, especialmente em comparação com o XML.

2.3. XML (Extensible Markup Language)

O XML é uma linguagem de marcação extensível que permite a representação de dados estruturados e metadados. É muito utilizado em cenários onde a estruturação e validação dos dados são essenciais, como documentos complexos e comunicações empresariais. No entanto, devido à sua complexidade e ao maior tamanho dos ficheiros gerados, o XML é frequentemente criticado por ser menos eficiente do que o JSON em termos de troca de dados orientada a objetos.

2.4. Serialização e Desserialização

A serialização é o processo de transformar objetos de dados num formato que possa ser transmitido ou armazenado, enquanto a desserialização converte esses dados de volta ao seu estado original. Ambos os processos são críticos em sistemas distribuídos, especialmente em microserviços onde a eficiência da comunicação entre diferentes serviços pode ser impactada pela escolha do formato de dados.

2.5. Eficiência em Microserviços

Em sistemas de microserviços, a **eficiência é medida pela rapidez com que os dados são serializados e desserializados**, bem como pelo tamanho dos ficheiros gerados. Formatos como o JSON costumam ser preferidos quando o desempenho é a prioridade, devido ao seu menor overhead e simplicidade. No entanto, o XML pode ser mais apropriado em cenários onde a estruturação e a integridade dos dados são cruciais, apesar de ser menos eficiente.

A Figura 1 abaixo ilustra o processo de serialização, transmissão e desserialização entre dois serviços num ambiente de microserviços. Neste exemplo, o Serviço A serializa os dados, envia-os pela rede (LAN), e o Serviço B desserializa esses dados, convertendo-os de volta ao seu estado original.

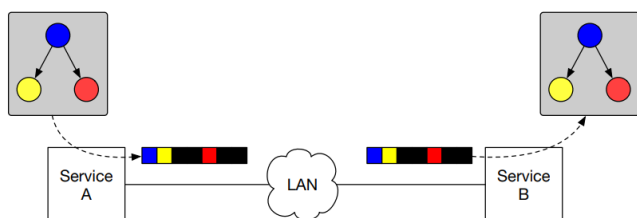


Figura 1 - Serialização, Transmissão e Desserialização

3. DECLARAÇÃO DO PROBLEMA

Nos sistemas que utilizam microserviços, a eficiência dos processos de serialização e desserialização de dados pode afetar de maneira significativa o desempenho e a escalabilidade do sistema. É importante selecionar o formato de dados apropriado para essas operações, especialmente em cenários que envolvem grande volume de trocas de dados.

Embora o JSON seja frequentemente usado devido à sua simplicidade e eficiência, o XML ainda prevalece em contextos que exigem maior estruturação de dados. O principal desafio deste trabalho é identificar qual dos dois formatos é mais eficiente num ambiente de microserviços, onde a otimização de recursos e a rapidez no processamento de dados são essenciais.

4. CONFIGURAÇÃO EXPERIMENTAL

Ambiente de desenvolvimento:

- ✓ **PC1:** IntelliJ IDEA 2024.2.3 (Ultimate Edition), JAVA 17.
- ✓ **PC2:** IntelliJ IDEA 2022.2.1 (Community Edition), JAVA 17.

Hardware:

- ✓ **PC1:** CPU Intel(R) Core(TM) Ultra 5 125H || RAM 32 GB || SSD || Windows 11 Home (64 bit)
- ✓ **PC2:** Intel(R) Core(TM) i5-8250U CPU 1.60GHz || RAM 8GB || SSD || Windows 11 Home (64 bit)

Configuração de software:

- ✓ **Jackson Databind** para serialização/desserialização JSON
 - **Versão:** 2.12.5
- ✓ **Jakarta XML Bind (JAXB)** para serialização/desserialização XML.
 - **Versão da API:** 3.0.0
 - **JAXB Runtime:** 3.0.2
 - **JAXB Implementation (Jakarta):** 3.0.2
 - **Jakarta Activation API:** 2.0.0

Estruturas dos testes: Utilizamos os testes para medir os tempos de serialização e desserialização de objetos, bem como o tamanho dos ficheiros gerados em cada iteração.

A estrutura dos dados usados nos testes foi baseada num objeto *User* com atributos simples (nome, idade e email), representados em JSON e XML.

Os testes foram realizados em 10.000 e 100.000 iterações para ambos os formatos (JSON e XML) e foram realizadas as seguintes medições:

- **Tempo de serialização:** O tempo necessário para converter o objeto *User* numa string no formato JSON ou XML.
- **Tempo de desserialização:** O tempo necessário para reconstruir o objeto *User* a partir da string serializada em JSON ou XML.
- **Tamanho dos dados gerados:** O tamanho em bytes da string gerada após a serialização para ambos os formatos.

Usamos a função `System.nanoTime()` do Java para medir os tempos de execução, e depois calculamos a média dos tempos. Os tamanhos dos ficheiros calculamos a partir do método `getBytes(StandardCharsets.UTF_8)` para garantir a consistência na codificação dos caracteres.

5. RESULTADOS

Realizamos testes com 100.000 iterações em dois computadores distintos para medir o tempo médio de serialização e desserialização, bem como o tamanho médio dos ficheiros gerados em JSON e XML. Os resultados que obtivemos estão apresentados na tabela abaixo:

Formato	Tempo Médio Serialização (ms)	Tempo Médio Desserialização (ms)	Tamanho Médio Ficheiro (bytes)
JSON PC1	0.0018	0.0017	64
XML PC1	0.0085	0.0499	142
JSON PC2	0.0080	0.0089	64
XML PC2	0.0128	0.1209	142

Tabela 1 - Tabela com os dados de 100.000 iterações

No PC1 o tempo médio de serialização do JSON foi de 0.0018ms, enquanto o tempo de serialização do XML foi de 0.0085ms, uma diferença notável à medida que o número de iterações aumentou. O tempo de desserialização apresentou uma diferença de desempenho ainda mais acentuada. O JSON desserializou em 0.0017ms, enquanto o XML levou 0.0499ms. Além disso, o tamanho médio dos ficheiros JSON foi significativamente menor (64 bytes), comparado ao tamanho médio dos ficheiros XML (142 bytes), o que reflete a simplicidade do JSON em comparação com a estrutura mais robusta e complexa do XML.

Em testes com menos iterações, como por exemplo 10.000 iterações, o XML tem menor tempo de serialização do que o JSON:

Formato	Tempo Médio Serialização (ms)	Tempo Médio Desserialização (ms)	Tamanho Médio Ficheiro (bytes)
JSON PC1	0.1447	0.0658	64
XML PC1	0.0758	0.4743	142
JSON PC2	0.2229	0.2134	64
XML PC2	0.1538	0.9500	142

Tabela 2 - Tabela com os dados de 10.000 iterações

6. DISCUSSÃO

Os resultados confirmam que o JSON é mais eficiente do que o XML em termos de tempo de desserialização e tamanho de ficheiros gerados, especialmente quando o número de iterações aumenta.

No entanto, observámos que, em iterações menores, o XML foi mais rápido na serialização. Esse comportamento pode ser atribuído ao overhead inicial do ambiente de execução, que afeta ambos os formatos, mas o XML, por ter uma estrutura mais padronizada e bem estabelecida, leva uma ligeira vantagem inicial. Além disso, o JSON depende de bibliotecas mais complexas que podem causar maior overhead nas primeiras execuções. No entanto, à medida que o volume de dados cresce e o número de iterações aumenta, o JSON supera o XML, tornando-se mais eficiente na

serialização, favorecendo o JSON pela sua simplicidade e menor complexidade estrutural.

O tempo de desserialização foi onde observámos diferenças mais significativas, com o JSON sendo quase 30 vezes mais rápido que o XML. Essa diferença é crucial para sistemas de microserviços, nos quais a velocidade de troca de dados impacta diretamente o desempenho.

Além disso, o tamanho dos ficheiros JSON foi significativamente menor, o que otimiza o uso da largura de banda em sistemas distribuídos. Em ambientes de microserviços, onde grandes volumes de dados são transmitidos continuamente, o tamanho reduzido do JSON minimiza o uso de largura de banda e melhora a velocidade de transmissão dos dados. A estrutura mais complexa do XML, com as tags de abertura e fechamento, resultou em ficheiros com mais que o dobro do tamanho dos ficheiros JSON.

7. CONCLUSÃO

Os resultados que obtivemos validam parcialmente a nossa hipótese de que o JSON é mais eficiente do que o XML em termos de tempo de serialização/desserialização e tamanho de ficheiros gerados, num cenário de microserviços.

Em iterações menores, observámos que o XML foi mais rápido na serialização, superando o JSON. No entanto, à medida que o número de iterações aumentou e o volume de dados se tornou maior, o JSON ultrapassou o XML nos tempos de serialização.

O JSON demonstrou uma vantagem clara em desserialização, e também gerou ficheiros significativamente menores, otimizando o uso de largura de banda em sistemas distribuídos mostrando ser a melhor opção para cenários de alta performance. Embora o XML tenha vantagens em cenários que exigem uma estrutura de dados mais robusta e detalhada, os tempos de desserialização são mais elevados e o maior tamanho dos ficheiros tornam-no menos eficiente para o contexto de microserviços. O JSON, por outro lado, mostrou-se bastante superior em termos de desempenho e eficiência de recursos, especialmente à medida que a quantidade de dados processados aumenta.

8. REFERÊNCIAS

- ✓ PowerPoints JSON e XML.
- ✓ <https://www.infocodify.com/web/serialize-and-deserialize-data>
- ✓ <https://medium.com/@truongbui95/serialization-and-deserialization-of-java-json-xml-libraries-42a1919d7f09>